

A CNN-Transformer Hybrid Architecture with Cross-Branch Feature Fusion for Multi-Organ Classification and Segmentation

Dong Liang¹, Lyuyang Tong^{2,3,4,5*}, and Bo Du^{2,3,4,5*}

¹ School of Computer Science, Hubei University of Technology, Wuhan, China

² School of Computer Science, Wuhan University, Wuhan, China

³ National Engineering Research Center for Multimedia Software, Wuhan University

⁴ Institute of Artificial Intelligence, School of Computer Science, Wuhan University

⁵ Hubei Key Laboratory of Multimedia and Network Communication Engineering, Wuhan University

* Team Leader's Name and Corresponding authors: Lyuyangtong@whu.edu.cn,
dubo@whu.edu.cn

Team Name: Sigma—WHU

Abstract. This paper proposes a CNN-Transformer hybrid architecture with cross-branch feature fusion for multi-organ classification and segmentation. We adopt a two-stage training strategy for the multi-task learning (MTL) framework. The model processes input images through two parallel but interacting branches: 1) a CNN branch employing hierarchical local feature extraction, and 2) a Transformer branch with blocks capturing global dependencies. Crucially, the branches exchange intermediate features through Cross-Branch Feature Fusion (CBFF) at multiple scales, enabling synergistic feature learning while preserving branch-specific characteristics. For segmentation tasks, our framework employs a dual-decoder strategy, and the final prediction is obtained by spatially weighting and combining the two outputs. For classification tasks, our framework employs a feature aggregation strategy; the fused features are pooled hierarchically (CNN's features + Transformer's features) before feeding into a single classifier. Auxiliary losses are applied to both branches during training.

Keywords: CNN-Transformer Hybrid Architecture · Multi-Organ Classification and Segmentation · Multi-task Learning.

1 Introduction

Medical ultrasound imaging has become indispensable in modern diagnostics due to its real-time capabilities, non-invasive nature, and cost-effectiveness [6][10]. Automated ultrasound image analysis confronts two major challenges: (1) struggling to effectively perform multi-task learning for Multi-Organ segmentation and classification, (2) lacking generalization ability across diverse anatomical structures and pathological presentations. Although deep learning has achieved

notable progress in medical image analysis, significant challenges remain in building systems that are robust to variations in institutions and examination protocols and reliable for clinical use. Most ultrasound-specific approaches typically utilize one of two methods: (1) pure convolutional architectures, such as U-Net variants, for local feature extraction, or (2) transformer-based models for global context modeling. The above methods fail to fully exploit the complementary strengths of both paradigms, particularly for multi-task scenarios involving both classification and segmentation. To address these challenges, we propose a CNN-Transformer hybrid architecture with cross-branch feature fusion for multi-organ classification and segmentation. The two-stage training strategy is adopted for the multi-task learning (MTL) framework. The hybrid architecture maintains simultaneous CNN and Transformer processing streams, preserving CNN’s local feature extraction (e.g., texture details) and Transformer’s global context modeling (e.g., anatomical relationships). Eventually, the Cross-Branch Feature Fusion (CBFF) dynamically fuses multi-scale features between branches through learnable attention weights, enabling adaptive local-global feature recombination.

2 Method

2.1 Multi-task Learning Paradigm

We adopt a two-stage training strategy for the multi-task learning (MTL) framework. In the **pre-training stage**, the model is trained on the complete dataset to acquire a general segmentation capability. In the **fine-tuning stage**, the pre-trained model is further adapted to the task-specific dataset to improve performance on the target domain. The procedure can be formulated as:

$$\theta^{(0)} = \arg \min_{\theta} \mathcal{L}_{\text{seg,cls}}(f_{\theta}(X_{\text{all}}), Y_{\text{all}}) \quad (\text{Pre-training}) \quad (1)$$

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{\text{seg,cls}}(f_{\theta}(X_{\text{task}}), Y_{\text{task}}), \text{ s.t. } \theta \leftarrow \theta^{(0)} \quad (\text{Fine-tuning}) \quad (2)$$

where $X_{\text{all}}, Y_{\text{all}}$ are the full dataset and labels, $X_{\text{task}}, Y_{\text{task}}$ are the task-specific dataset and labels, $\theta^{(0)}$ are the pre-trained parameters, θ^* are the fine-tuned parameters, and $\mathcal{L}_{\text{seg}}$ denotes the segmentation loss (Dice), or classification loss (Cross Entropy). Our framework leverages the **multi-task learning** paradigm to jointly optimize classification and segmentation within this two-stage process. Unlike single-task pipelines that require training separate models, our MTL design offers the following:

- **Shared Feature Extraction:** A unified encoder, initialized with pre-trained parameters $\theta^{(0)}$, is shared across both tasks, ensuring consistent feature representations while reducing the overall computational cost.
- **Task-Specific Processing:** Two dedicated decoders operate on the shared features:
 - **Segmentation branch:** A U-Net style decoder with skip connections, optimized via $\mathcal{L}_{\text{seg}}$ in both stages.
 - **Classification branch:** A hierarchical feature pooling mechanism, trained alongside segmentation in the fine-tuning stage.

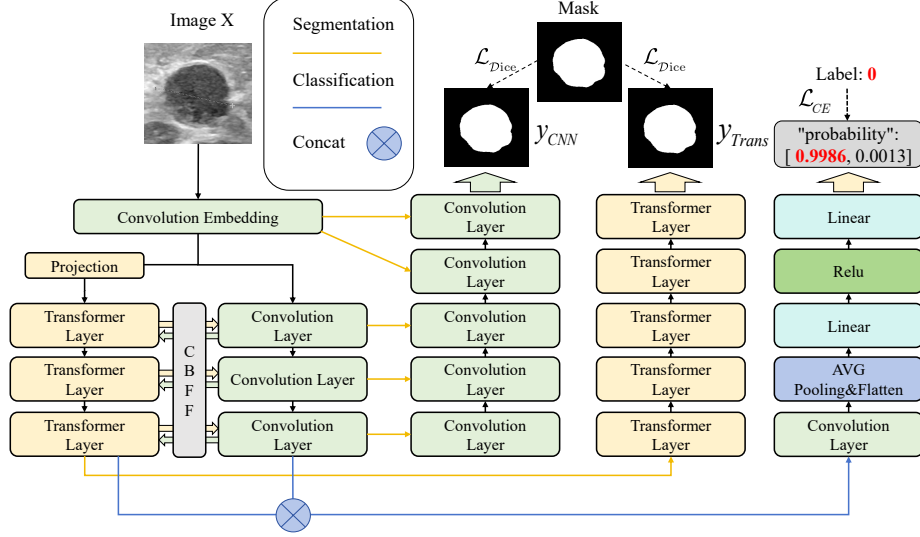


Fig. 1. The framework of the CNN-Transformer Hybrid Architecture with Cross-Branch Feature Fusion

2.2 Model Architectures

The framework of the CNN-Transformer Hybrid Architecture with Cross-Branch Feature Fusion is illustrated in Fig. 1. The CNN branch extracts local features via convolutional operations:

$$\mathbf{F}_i^{\text{cnn}} = \begin{cases} \mathbf{X}, & i = 0, \\ \text{Conv}(\mathbf{F}_{i-1}^{\text{cnn}}), & i > 0 \end{cases} \quad (3)$$

while the Transformer branch models global dependencies. The input is first mapped into patch embeddings:

$$\mathbf{F}_0^{\text{trans}} = \text{PatchEmbed}(\mathbf{X}) \in R^{N \times D}, \quad (4)$$

and updated through multi-head self-attention (MHSA) and MLP layers:

$$\mathbf{F}_i^{\text{trans}} = \mathbf{F}_{i-1}^{\text{trans}} + \text{MHSA}(\text{LN}(\mathbf{F}_{i-1}^{\text{trans}})), \quad \mathbf{F}_i^{\text{trans}} \leftarrow \mathbf{F}_{i-1}^{\text{trans}} + \text{MLP}(\text{LN}(\mathbf{F}_i^{\text{trans}})), \quad (5)$$

where i denotes the layer index in each branch, N is the number of patches, and D is the embedding dimension.

The Cross-Branch Feature Fusion (CBFF) is designed to integrate features from the CNN branch and the Transformer branch, enabling effective multi-scale feature fusion. Let $\mathbf{F}_i^{\text{cnn}}$ denote the local feature map extracted by the CNN branch at layer i through convolutional operations, and $\mathbf{F}_i^{\text{trans}}$ denote the global contextual representation obtained by the Transformer branch at layer

i via patch embedding and self-attention. To address the dimensional disparity between $\mathbf{F}_{\text{cnn}}$ and $\mathbf{F}_{\text{trans}}$, we first perform a feature alignment operation:

$$\tilde{\mathbf{F}}_i^{\text{cnn}} = \phi_{\text{cnn}}(\mathbf{F}_i^{\text{cnn}}), \quad \tilde{\mathbf{F}}_i^{\text{trans}} = \phi_{\text{trans}}(\mathbf{F}_i^{\text{trans}}) \quad (6)$$

where $\phi_{\text{cnn}}(\cdot)$ and $\phi_{\text{trans}}(\cdot)$ are alignment functions composed of convolutional layers, pooling, normalization, and activation functions, ensuring both features share the same spatial resolution and channel dimension. The aligned features are then fused by element-wise addition:

$$\mathbf{F}_i^{\text{fused}} = \tilde{\mathbf{F}}_i^{\text{cnn}} + \tilde{\mathbf{F}}_i^{\text{trans}} \quad (7)$$

This operation combines the locally captured patterns from the CNN with the global contextual relationships from the Transformer. The CBFF output $\mathbf{F}_{\text{fused}}$ is fed into subsequent Transformer and CNN blocks to achieve bidirectional feature sharing. For the Transformer path, the fused features are added to the token embeddings of the next Transformer block:

$$\mathbf{F}_{i+1}^{\text{trans}} = \text{TransformerLayer}(\mathbf{F}_i^{\text{fused}}) \quad (8)$$

Similarly, for the CNN path:

$$\mathbf{F}_{i+1}^{\text{cnn}} = \text{CNNLayer}(\mathbf{F}_i^{\text{fused}}) \quad (9)$$

where i denotes the layer index, and $i + 1$ represents the next layer. Through these operations, CBFF enables seamless integration of CNN and Transformer representations, enhancing the model’s ability to capture intricate patterns and contextual relationships.

For the segmentation task, the CNN decoder follows a structure similar to that of U-Net. At each decoding stage, the output feature map is concatenated with the feature map from the last convolutional layer of the corresponding encoder stage, enabling the integration of both high-level semantic and low-level spatial information. The stem module also includes three convolutional blocks to extract the features needed for the decoding process. In contrast to the U-Net decoder, our Transformer-based decoder performs upsampling on global representations, as the spatial resolution of the embeddings remains constant across all Transformer encoder layers.

For the classification task, the decoder integrates features from both the Transformer and CNN branches. Specifically, the final output feature map from the Transformer branch, denoted as $\mathbf{F}_{\text{Trans}}$, is concatenated with the final output feature map from the CNN branch, denoted as $\mathbf{F}_{\text{CNN}}$, to form a joint representation:

$$\mathbf{F}_{\text{joint}} = \text{Concat}(\mathbf{F}_{\text{Trans}}, \mathbf{F}_{\text{CNN}}). \quad (10)$$

The fused feature $\mathbf{F}_{\text{joint}}$ is then processed by a convolutional layer to refine spatial and channel-wise information:

$$\mathbf{F}_{\text{conv}} = \sigma(\mathbf{F}_{\text{joint}} * \mathbf{W}_{\text{conv}}), \quad (11)$$

where $*$ denotes convolution and $\sigma(\cdot)$ is a nonlinear activation function (e.g., ReLU). Subsequently, global average pooling is applied to reduce spatial dimensions:

$$\mathbf{f}_{\text{gap}} = \text{GAP}(\mathbf{F}_{\text{conv}}), \quad (12)$$

followed by a fully connected (linear) layer, an activation, and another linear layer for final classification:

$$\hat{\mathbf{y}} = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{f}_{\text{gap}} + \mathbf{b}_1) + \mathbf{b}_2. \quad (13)$$

where $\phi(\cdot)$ denotes the activation function and the output dimension of $\hat{\mathbf{y}}$ depends on the number of categories in the classification task.

2.3 Loss Function

Our model handles both segmentation and classification tasks, and the loss function is designed accordingly.

Segmentation Loss. For segmentation tasks, we adopt the **Dice loss** to measure the similarity between the predicted mask and the ground truth mask. Both the CNN branch and the Transformer branch produce segmentation predictions, denoted as $\hat{\mathbf{M}}_{\text{CNN}}$ and $\hat{\mathbf{M}}_{\text{Trans}}$, respectively. The Dice loss for each branch is defined as:

$$\mathcal{L}_{\text{Dice}}(\hat{\mathbf{M}}, \mathbf{M}) = 1 - \frac{2 \sum_i \hat{M}_i M_i + \epsilon}{\sum_i \hat{M}_i + \sum_i M_i + \epsilon}, \quad (14)$$

where $\mathbf{M}$ is the ground truth mask, $\hat{\mathbf{M}}$ is the predicted mask, i indexes all pixels, and ϵ is a small constant to avoid division by zero. The total segmentation loss is a weighted sum of the Dice losses from both branches:

$$\mathcal{L}_{\text{seg}} = \frac{\mathcal{L}_{\text{Dice}}(\mathbf{y}_{\text{CNN}}, \mathbf{M}) + \mathcal{L}_{\text{Dice}}(\mathbf{y}_{\text{Trans}}, \mathbf{M})}{2}. \quad (15)$$

Classification Loss. For classification tasks, we adopt the standard **cross-entropy loss**. The classification loss is:

$$\mathcal{L}_{\text{cls}} = - \sum_{c=1}^C y_c \log \text{softmax}(\hat{y}_c), \quad (16)$$

where $\hat{\mathbf{y}}$ denote the predicted logits from the classification decoder and $\mathbf{y}$ the one-hot ground truth label vector, C is the number of classes.

Total Loss. The overall loss of the model depends on the task:

$$\mathcal{L}_{\text{total}} = \begin{cases} \mathcal{L}_{\text{seg}}, & \text{for segmentation tasks,} \\ \mathcal{L}_{\text{cls}}, & \text{for classification tasks.} \end{cases} \quad (17)$$

This design allows the model to jointly optimize the CNN and Transformer branches for segmentation, while using standard cross-entropy for classification.

3 Experimental Results

3.1 Implementation Details

For our experiments, we used a combined dataset provided for the challenge, which consists of multiple public ultrasound datasets [2][11][3][1][9][8][4][5][7] as well as several private datasets collected from three hospitals. The dataset includes images with either classification labels, segmentation masks, or both. For cases derived from video sequences, frames from the same patient share the same label. This diverse dataset allows evaluation of both segmentation and classification tasks, covering multiple organ sites and tumor types. Prior to training, all images were preprocessed to remove any user interface (UI) elements, preventing the model from being biased by irrelevant visual cues. Data augmentation was applied to improve model robustness, including random rotations, horizontal and vertical flips, and other geometric transformations. All images were then resized to a uniform resolution of 256×256 pixels.

For model optimization, the initial learning rate was set to 1×10^{-4} , and the batch size was 16. Training was conducted in two stages:

1. **Pre-training stage:** the model was trained on the combined dataset across all sources for 300 epochs to improve generalization across diverse imaging conditions.
2. **Fine-tuning stage:** the model was further trained for 150 epochs on each individual dataset to better adapt to the specific characteristics and distributions of the corresponding task.

This two-stage training strategy ensures that the model learns both generalizable features and dataset-specific patterns, enhancing its performance on segmentation and classification tasks.

3.2 Results

The visualization results of the model on the validation set are presented in Fig. 2, and the detailed quantitative metrics are listed in Table 1.

Table 1. Segmentation (T1) and Classification (T2) results on validation datasets

Task 1: Segmentation					
Breast	Cardiac	Thyroid	Fetal Head	Kidney	Average
1.03	0.93	0.75	1.06	0.99	0.93
Task 2: Classification					
Breast B/M	Breast Subtype	Appendix	Liver	Average	
0.73	0.55	0.74	0.85	0.72	

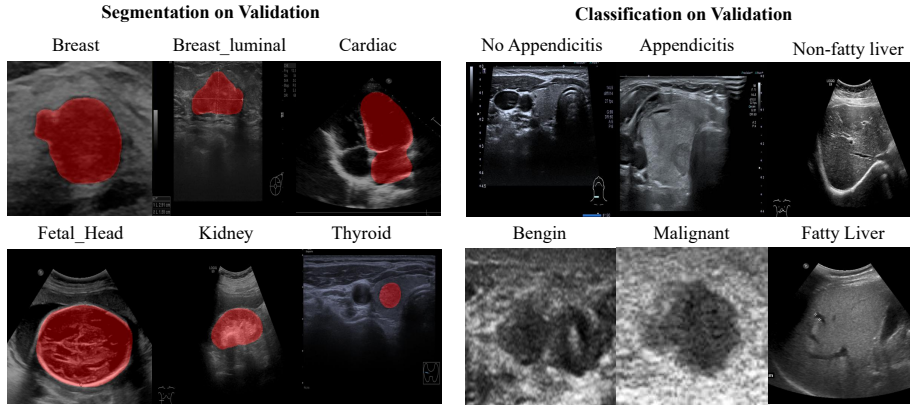


Fig. 2. Model Prediction on Validation Sample

4 Discussion and Conclusion

In conclusion, we present a CNN-Transformer hybrid framework with cross-branch feature fusion that effectively leverages both local and global representations for multi-organ classification and segmentation. The CNN and Transformer branches exchange intermediate features through Cross-Branch Feature Fusion (CBFF) at multiple scales, allowing each branch to benefit from the other’s information while retaining its own unique characteristics. Experimental results demonstrate that the proposed approach produces accurate segmentation and reliable classification, highlighting the benefits of cross-branch interaction and multi-scale feature fusion. These results indicate the framework’s potential for broader applications in medical image analysis and other multi-task visual learning scenarios.

5 Acknowledgements

This work was supported in part by the National Key Research and Development Program of China under Grants 2023YFC2705700, the National Natural Science Foundation of China under Grants 62306217 and 62225113, the Post-doctoral Fellowship Program of CPSF under Grant Number GZC20231987, the China Postdoctoral Science Foundation under Grant Number 2024T170686 and 2024M752471, the Major Program (JD) of Hubei Province (2023BAA017), the Innovative Research Group Project of Hubei Province under Grants 2024AFA017. The numerical calculations in this paper have been done on the supercomputing system in the Supercomputing Center of Wuhan University.

References

- [1] Michał Byra et al. “Transfer learning with deep convolutional neural network for liver steatosis assessment in ultrasound images”. In: *International journal of computer assisted radiology and surgery* 13.12 (2018), pp. 1895–1903.
- [2] Walid Al-Dhabyani et al. “Dataset of breast ultrasound images”. In: *Data in brief* 28 (2020), p. 104863.
- [3] Wilfrido Gómez-Flores, Maria Julia Gregorio-Calas, and Wagner Coelho de Albuquerque Pereira. “BUS-BRA: a breast ultrasound dataset for assessing computer-aided diagnosis systems”. In: *Medical Physics* 51.4 (2024), pp. 3110–3123.
- [4] Thomas LA van den Heuvel et al. “Automated measurement of fetal head circumference using 2D ultrasound images”. In: *PloS one* 13.8 (2018), e0200412.
- [5] Sarah Leclerc et al. “Deep learning for segmentation using an open large-scale dataset in 2D echocardiography”. In: *IEEE transactions on medical imaging* 38.9 (2019), pp. 2198–2210.
- [6] Zehui Lin et al. “UniUSNet: A promptable framework for universal ultrasound disease prediction and tissue segmentation”. In: *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE. 2024, pp. 3501–3504.
- [7] Ričards Marcinkevičs et al. “Regensburg pediatric appendicitis dataset”. In: *(No Title)* (2023).
- [8] Lina Pedraza et al. “An open access thyroid ultrasound image database”. In: *10th International symposium on medical information processing and analysis*. Vol. 9287. SPIE. 2015, pp. 188–193.
- [9] Rohit Singla et al. “The open kidney ultrasound data set”. In: *International Workshop on Advances in Simplifying Medical Ultrasound*. Springer. 2023, pp. 155–164.
- [10] Tianyu Zhang et al. “Predicting breast cancer types on and beyond molecular level in a multi-modal fashion”. In: *NPJ breast cancer* 9.1 (2023), p. 16.
- [11] Yingtao Zhang et al. “BUSIS: a benchmark for breast ultrasound image segmentation”. In: *Healthcare*. Vol. 10. 4. MDPI. 2022, p. 729.